UNIT 1

1. Stack Algorithms

push :

Step 1: If top == MAX - 1

Print "Stack Overflow"

Exit

Step 2: top = top + 1

Step 3: stack[top] = item

Step 4: Exit

pop:

Step 1: If top == -1

Print "Stack Underflow"

Exit

Step 2: item = stack[top]

Step 3: top = top - 1

Step 4: Return item

peek:

Step 1: If top == -1

Print "Stack is Empty"

Exit

Step 2: Return stack[top]

2. Tower of Honoi

Step 1: If n == 1

Move disk 1 from Source to Destination

Return

Step 2: Call ToH(n - 1, Source, Destination, Auxiliary)

Step 3: Move disk n from Source to Destination

Step 4: Call ToH(n - 1, Auxiliary, Source, Destination)

Step 5: Stop

3. Recursion Algo

Step 1: If base condition is satisfied

Return result

Step 2: Else

Call RECURSIVE_FUNCTION(smaller value of n)

Step 3: Combine result (if required)

Step 4: Return final result

4. Infix to Postfix

Step 1: Initialize empty stack S and empty postfix string P

Step 2: Scan infix expression from left to right

Step 3: If operand, add it to P

Step 4: If '(' push to stack

Step 5: If ')', pop from stack and add to P until '(' is found

Step 6: If operator:

While stack not empty and precedence(top) ≥ precedence(current)

Pop from stack and add to P

Push current operator to stack

Step 7: After scanning, pop all operators from stack to P

Step 8: Return postfix expression P

5. Infix to Prefix

Step 1: Initialize an empty stack S (for operators)

Step 2: Initialize an empty string P (for prefix expression)

Step 3: Scan the infix expression from right to left

Step 4: If the scanned symbol is an operand

Add it to the beginning of P

Step 5: If the scanned symbol is ')'

Push it onto stack S

Step 6: If the scanned symbol is '('

Pop operators from stack S and add to beginning of P

until ')' is found

Pop and discard ')'

Step 7: If the scanned symbol is an operator

While stack is not empty AND

precedence(top of S) > precedence(current operator)

OR

precedence(top of S) == precedence(current operator)

AND operator is left associative

Pop from stack S and add to beginning of P

Push current operator onto stack S

Step 8: After scanning the entire expression

Pop all remaining operators from stack S

and add them to the beginning of P

Step 9: Return prefix expression P

6. Algorithm: PREFIX_TO_POSTFIX(expression)

Step 1: Initialize empty stack S

Step 2: Scan prefix expression from right to left

Step 3: If operand, push onto stack

Step 4: If operator:

Pop operand1

Pop operand2

Create string = operand1 + operand2 + operator

Push string back to stack

Step 5: Final element in stack is postfix expression

7. Algorithm: PREFIX_TO_INFIX(expression)

Step 1: Initialize empty stack S

Step 2: Scan prefix expression from right to left

Step 3: If operand, push onto stack

Step 4: If operator:

Pop operand1

Pop operand2

Create string = (operand1 operator operand2)

Push string back to stack

Step 5: Final element in stack is infix expression

8. Algorithm: POSTFIX_TO_PREFIX(expression)

Step 1: Initialize empty stack S

Step 2: Scan postfix expression from left to right

Step 3: If operand, push onto stack

Step 4: If operator:

Pop operand2

Pop operand1

Create string = operator + operand1 + operand2

Push string back to stack

Step 5: Final element in stack is prefix expression

9. Algorithm: POSTFIX_TO_INFIX(expression)

Step 1: Initialize empty stack S

Step 2: Scan postfix expression from left to right

Step 3: If operand, push onto stack

Step 4: If operator:

Pop operand2

Pop operand1

Create string = (operand1 operator operand2)

Push string back to stack

Step 5: Final element in stack is infix expression

UNIT 2

1. FOR Queue

A. Algorithm Name: ENQUEUE(queue, n, item)

Step 1: If rear == n − 1
        Print "Queue Overflow"
        Exit
Step 2: If front == -1
        front = 0
Step 3: rear = rear + 1
Step 4: queue[rear] = item
Step 5: Exit

B. Algorithm Name: DEQUEUE(queue)

Step 1: If front == -1 OR front > rear
        Print "Queue Underflow"
        Exit
Step 2: item = queue[front]
Step 3: front = front + 1
Step 4: Return item

C. Algorithm Name: DISPLAY(queue)

Step 1: If front == -1 OR front > rear
        Print "Queue is Empty"
        Exit
Step 2: For i = front to rear
        Print queue[i]
Step 3: Exit

D. CIRCULAR_ENQUEUE(queue, n, item)

Step 1: If (rear + 1) % n == front
        Print "Queue Overflow"
        Exit
Step 2: If front == -1
        front = 0
        rear = 0
      Else

```
        rear = (rear + 1) % n
Step 3: queue[rear] = item
Step 4: Exit
```

E. CIRCULAR_DEQUEUE(queue, n)

```
Step 1: If front == -1
        Print "Queue Underflow"
        Exit
Step 2: item = queue[front]
Step 3: If front == rear
        front = -1
        rear = -1
      Else
        front = (front + 1) % n
Step 4: Return item
```

INSERT_FRONT(deque, n, item)

```
Step 1: If front == 0
        Print "Overflow at Front"
        Exit
Step 2: If front == -1
        front = 0
        rear = 0
      Else
        front = front − 1
Step 3: deque[front] = item
Step 4: Exit
```

INSERT_REAR(deque, n, item)

```
Step 1: If rear == n − 1
        Print "Overflow at Rear"
        Exit
Step 2: If front == -1
        front = 0
        rear = 0
      Else
        rear = rear + 1
Step 3: deque[rear] = item
Step 4: Exit
```

DELETE_FRONT(deque)

```
Step 1: If front == -1
          Print "Underflow"
          Exit
Step 2: item = deque[front]
Step 3: If front == rear
          front = -1
          rear = -1
        Else
          front = front + 1
Step 4: Return item
```

DELETE_REAR(deque)

```
Step 1: If rear == -1
          Print "Underflow"
          Exit
Step 2: item = deque[rear]
Step 3: If front == rear
          front = -1
          rear = -1
        Else
          rear = rear − 1
Step 4: Return item
```

F. PRIORITY_ENQUEUE(queue, n, item, priority)

```
Step 1: If rear == n − 1
          Print "Queue Overflow"
          Exit
Step 2: If front == -1
          front = 0
          rear = 0
          queue[rear] = (item, priority)
          Exit
Step 3: Set i = rear
Step 4: While i >= front AND queue[i].priority < priority
          queue[i + 1] = queue[i]
          i = i − 1
Step 5: queue[i + 1] = (item, priority)
Step 6: rear = rear + 1
Step 7: Exit
```

PRIORITY_DEQUEUE(queue)

Step 1: If front == -1 OR front > rear
        Print "Queue Underflow"
        Exit
Step 2: item = queue[front]
Step 3: front = front + 1
Step 4: Return item

PRIORITY_DISPLAY(queue)

Step 1: If front == -1 OR front > rear
        Print "Queue is Empty"
        Exit
Step 2: For i = front to rear
        Print queue[i].data, queue[i].priority
Step 3: Exit

2. linked list

A. **Algorithm: INSERT_BEGIN_SLL(head, item)**

Step 1: Create new node N
Step 2: N.data = item
Step 3: N.next = head
Step 4: head = N
Step 5: Stop

B. **Algorithm: INSERT_END_SLL(head, item)**

Step 1: Create new node N
Step 2: N.data = item
Step 3: N.next = NULL
Step 4: If head == NULL
        head = N
        Stop
Step 5: Set temp = head
Step 6: While temp.next ≠ NULL
        temp = temp.next
Step 7: temp.next = N
Step 8: Stop

C. Algorithm: DELETE_BEGIN_SLL(head)

Step 1: If head == NULL
        Print "List Empty"
        Stop
Step 2: temp = head
Step 3: head = head.next
Step 4: Delete temp
Step 5: Stop

D. Algorithm: DELETE_END_SLL(head)

Step 1: If head == NULL
        Print "List Empty"
        Stop
Step 2: If head.next == NULL
        Delete head
        head = NULL
        Stop
Step 3: Set temp = head
Step 4: While temp.next.next ≠ NULL
        temp = temp.next
Step 5: Delete temp.next
Step 6: temp.next = NULL
Step 7: Stop

Doubly linked list

E. Algorithm: INSERT_BEGIN_DLL(head, item)

Step 1: Create new node N
Step 2: N.data = item
Step 3: N.prev = NULL
Step 4: N.next = head
Step 5: If head ≠ NULL
        head.prev = N
Step 6: head = N
Step 7: Stop

F. **Algorithm: INSERT_END_DLL(head, item)**

Step 1: Create new node N
Step 2: N.data = item
Step 3: N.next = NULL
Step 4: If head == NULL
    N.prev = NULL
    head = N
    Stop
Step 5: temp = head
Step 6: While temp.next ≠ NULL
    temp = temp.next
Step 7: temp.next = N
Step 8: N.prev = temp
Step 9: Stop

G. **Algorithm: DELETE_BEGIN_DLL(head)**

Step 1: If head == NULL
    Print "List Empty"
    Stop
Step 2: temp = head
Step 3: head = head.next
Step 4: If head ≠ NULL
    head.prev = NULL
Step 5: Delete temp
Step 6: Stop

H. **Algorithm: DELETE_END_DLL(head)**

Step 1: If head == NULL
    Print "List Empty"
    Stop
Step 2: temp = head
Step 3: While temp.next ≠ NULL
    temp = temp.next
Step 4: If temp.prev ≠ NULL
    temp.prev.next = NULL
  Else
    head = NULL
Step 5: Delete temp
Step 6: Stop

3. Circular linked list

A. **Algorithm: INSERT_BEGIN_CLL(head, item)**

Step 1: Create new node N
Step 2: N.data = item
Step 3: If head == NULL
        N.next = N
        head = N
        Stop
Step 4: temp = head
Step 5: While temp.next ≠ head
        temp = temp.next
Step 6: N.next = head
Step 7: temp.next = N
Step 8: head = N
Step 9: Stop

B. **Algorithm: INSERT_END_CLL(head, item)**

Step 1: Create new node N
Step 2: N.data = item
Step 3: If head == NULL
        N.next = N
        head = N
        Stop
Step 4: temp = head
Step 5: While temp.next ≠ head
        temp = temp.next
Step 6: temp.next = N
Step 7: N.next = head
Step 8: Stop

C. **Algorithm: DELETE_BEGIN_CLL(head)**

Step 1: If head == NULL
        Print "List Empty"
        Stop
Step 2: If head.next == head
        Delete head
        head = NULL
        Stop
Step 3: temp = head

Step 4: last = head
Step 5: While last.next ≠ head
        last = last.next
Step 6: head = temp.next
Step 7: last.next = head
Step 8: Delete temp
Step 9: Stop

D. **Algorithm: DELETE_END_CLL(head)**

Step 1: If head == NULL
        Print "List Empty"
        Stop
Step 2: If head.next == head
        Delete head
        head = NULL
        Stop
Step 3: temp = head
Step 4: While temp.next.next ≠ head
        temp = temp.next
Step 5: Delete temp.next
Step 6: temp.next = head
Step 7: Stop

E. Algorithm: INSERT_MIDDLE_CLL(head, item, pos)

Step 1: Create new node NStep 2: N.data = item
Step 3: If head == NULL
        Print "List Empty"
        Stop
Step 4: If pos == 1
        (Insert at beginning using INSERT_BEGIN_CLL)
        Stop
Step 5: Set temp = headStep 6: For i = 1 to pos − 2
        temp = temp.next
        If temp == head
           Print "Invalid Position"
           Stop
Step 7: N.next = temp.nextStep 8: temp.next = NStep 9: Stop

F. Algorithm: DELETE_MIDDLE_CLL(head, pos)

Step 1: If head == NULL
Print "List Empty"
Stop
Step 2: If pos == 1
(Delete at beginning using DELETE_BEGIN_CLL)
Stop
Step 3: Set temp = headStep 4: For i = 1 to pos − 2
temp = temp.next
If temp.next == head
Print "Invalid Position"
Stop
Step 5: del = temp.nextStep 6: temp.next = del.nextStep 7: Delete delStep 8: Stop

UNIT 3

1. Linear search algo

Linear Search (Array A, Value x)

Step 1: Set i to 1

Step 2: if i > n then go to step 7

Step 3: if A[i] = x then go to step 6

Step 4: Set i to i + 1

Step 5: Go to Step 2

Step 6: Print Element x Found at index i and go to step 8

Step 7: Print element not found

Step 8: Exit Pseudocode

procedure linear_search (list, value)

2. Binary search algo

Step 1: Set low = 0

Step 2: Set high = n − 1

Step 3: While low ≤ high

mid = (low + high) / 2

If arr[mid] == key

Return mid (element found)

Else if key < arr[mid]

high = mid − 1

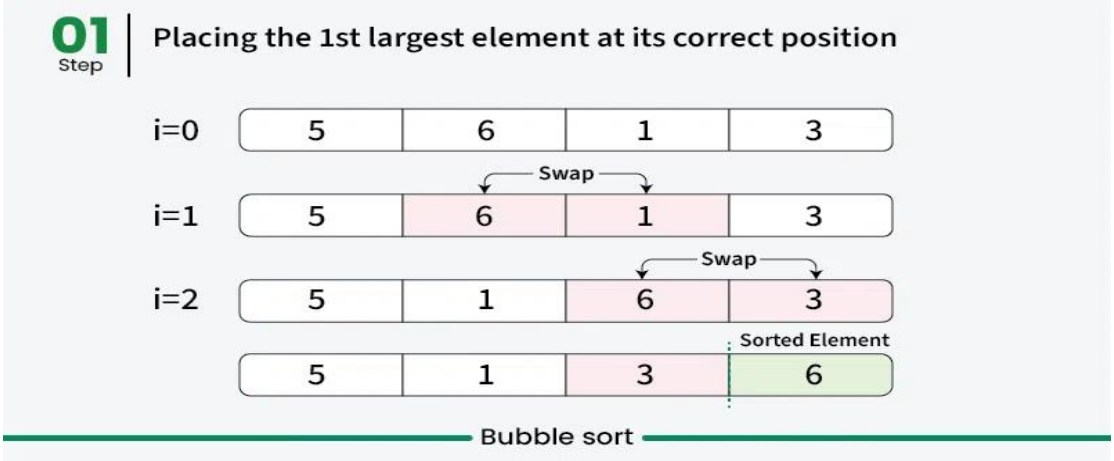
Else

low = mid + 1

Step 4: If loop ends

Return -1 (element not found)

3. Bubble sort



“Compare, swap, repeat — biggest goes last”

Step 1: For i = 0 to n − 2

Step 2: For j = 0 to n − i − 2

Step 3: If arr[j] > arr[j + 1]

Step 4: Swap arr[j] and arr[j + 1]

Step 5: Stop

4. Selection sort

“Select minimum and swap”

Step 1: For i = 0 to n − 2

Step 2: Set min = i

Step 3: For j = i + 1 to n − 1

Step 4: If arr[j] < arr[min]

Step 5: min = j

Step 6: Swap arr[i] and arr[min]

Step 7: Stop

5. Insertion Sort

“Shift bigger, insert smaller”

Step 1: For i = 1 to n − 1

Step 2: key = arr[i]

Step 3: j = i − 1

Step 4: While j ≥ 0 AND arr[j] > key

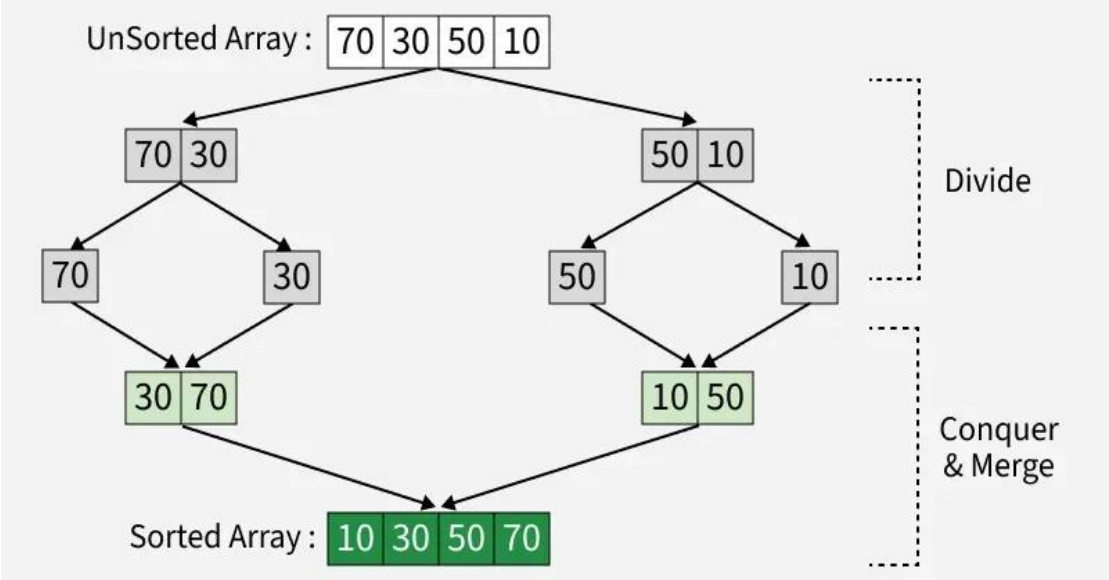
Step 5: arr[j + 1] = arr[j]

Step 6: j = j − 1

Step 7: arr[j + 1] = key

Step 8: Stop

6. Merge sort



Divide → Sort → Merge

Step 1: Create temporary arrays L and R

Step 2: Copy arr[low..mid] into L

Step 3: Copy arr[mid+1..high] into R

Step 4: Set i = 0, j = 0, k = low

Step 5: While i < size(L) and j < size(R)

If L[i] ≤ R[j]

arr[k] = L[i]; i++

Else

arr[k] = R[j]; j++

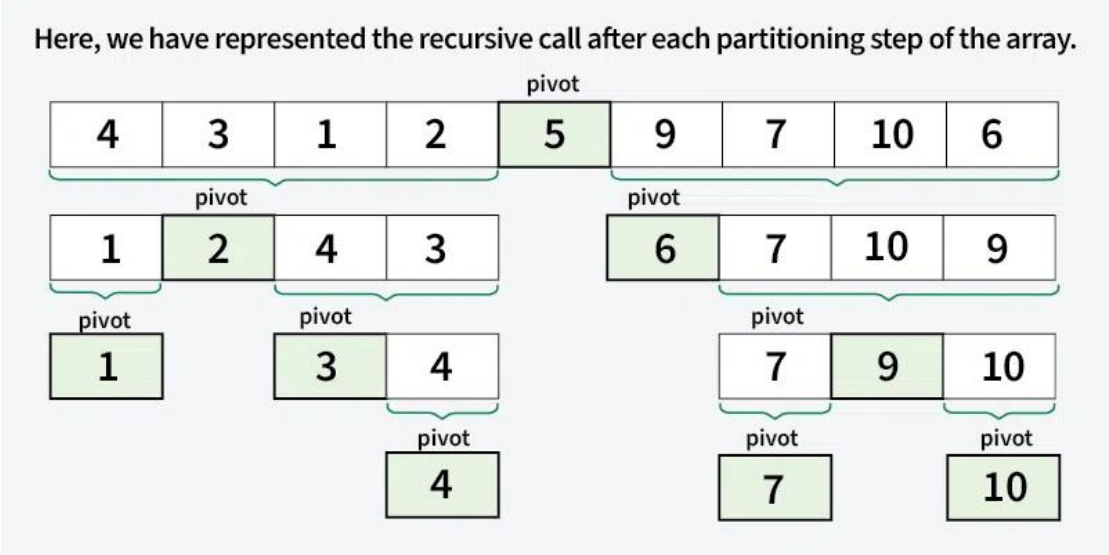
k++

Step 6: Copy remaining elements of L (if any)

Step 7: Copy remaining elements of R (if any)

Step 8: Stop

7. Quick sort



"Choose pivot → partition → sort left & right"

Algorithm Name: QUICK_SORT(arr, low, high)

Step 1: If low < high

Step 2: pivotPos = PARTITION(arr, low, high)

Step 3: Call QUICK_SORT(arr, low, pivotPos − 1)

Step 4: Call QUICK_SORT(arr, pivotPos + 1, high)

Step 5: Stop

Algorithm Name: PARTITION(arr, low, high)

Step 1: Select pivot = arr[high]

Step 2: Set i = low − 1

Step 3: For j = low to high − 1

If arr[j] ≤ pivot

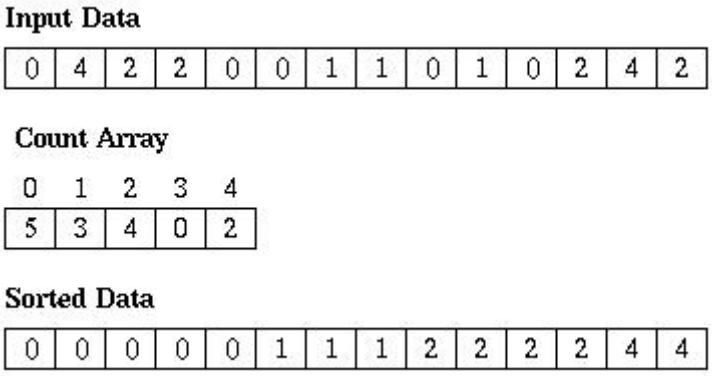
i = i + 1

Swap arr[i] and arr[j]

Step 4: Swap arr[i + 1] and arr[high]

Step 5: Return (i + 1) as pivot position

8. Counting Sort



“Count → Add → Place”

Step 1: Create a count array count[0…k] and initialize all values to 0

Step 2: For i = 0 to n − 1

count[arr[i]] = count[arr[i]] + 1

Step 3: For i = 1 to k

count[i] = count[i] + count[i − 1]

Step 4: Create output array output[n]

Step 5: For i = n − 1 down to 0

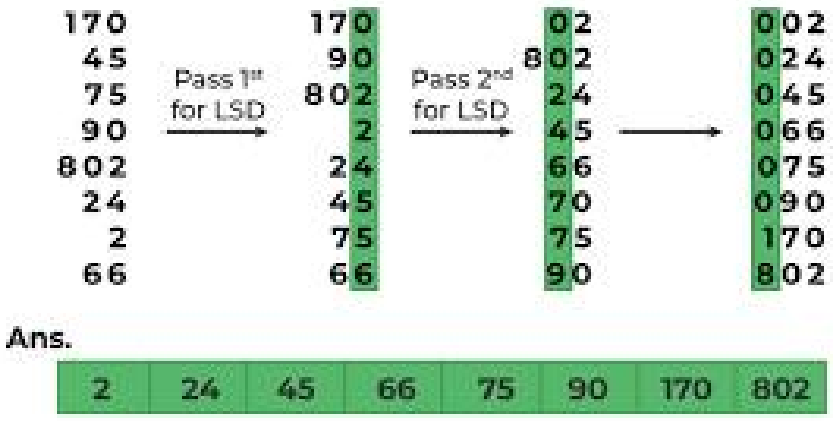
output[count[arr[i]] − 1] = arr[i]

count[arr[i]] = count[arr[i]] − 1

Step 6: Copy output array to original array arr

Step 7: Stop

9. Redix sort



“Sort by digits, one place at a time”

Algorithm Name: RADIX_SORT(arr, n)

Step 1: Find the maximum number max in arr

Step 2: For exp = 1; max/exp > 0; exp = exp × 10

Call COUNTING_SORT(arr, n, exp)

Step 3: Stop

Algorithm Name: COUNTING_SORT(arr, n, exp)

Step 1: Create count[0…9] and initialize to 0

Step 2: For i = 0 to n − 1

digit = (arr[i] / exp) % 10

count[digit]++

Step 3: For i = 1 to 9

count[i] = count[i] + count[i − 1]

Step 4: Create output array output[n]

Step 5: For i = n − 1 down to 0

digit = (arr[i] / exp) % 10

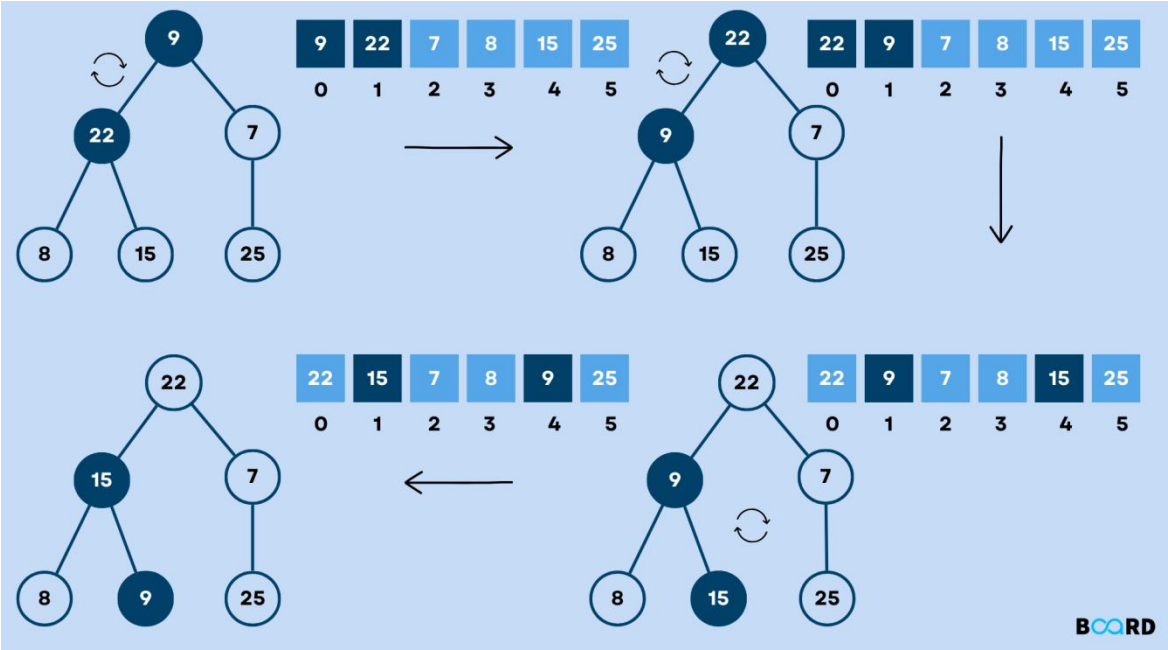
output[count[digit] − 1] = arr[i]

count[digit]--

Step 6: Copy output array to arr

Step 7: Stop

10. Heap sort



"Build heap → remove max → repeat"

Algorithm Name: HEAP_SORT(arr, n)

Step 1: Build a Max Heap from the array

Step 2: For i = n − 1 down to 1

Swap arr[0] and arr[i]

Reduce heap size by 1

Call HEAPIFY(arr, i, 0)

Step 3: Stop

Algorithm Name: HEAPIFY(arr, n, i)

Step 1: Set largest = i

Step 2: left = 2i + 1

Step 3: right = 2i + 2

Step 4: If left < n AND arr[left] > arr[largest]

largest = left

Step 5: If right < n AND arr[right] > arr[largest]

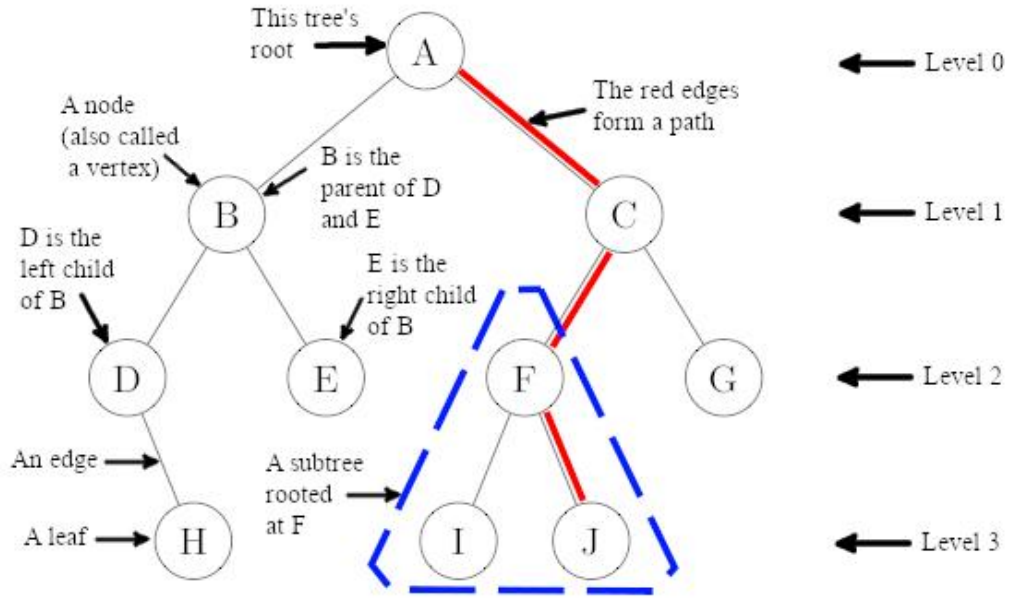
largest = right

Step 6: If largest ≠ i

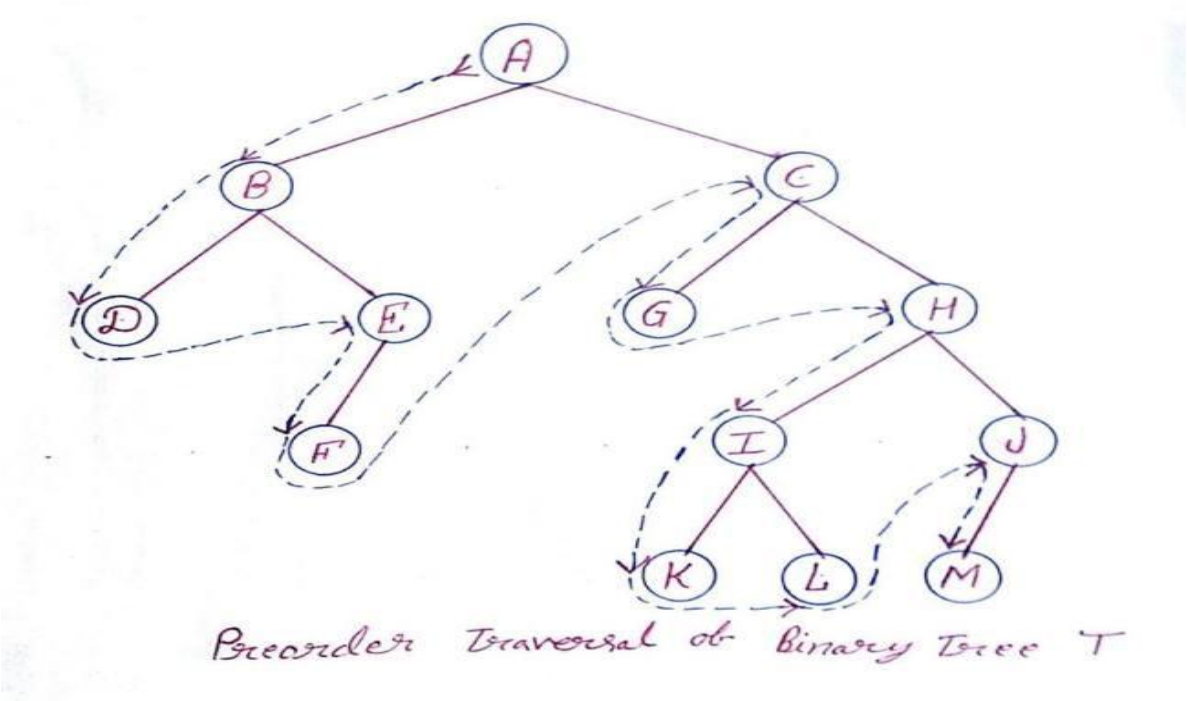
Swap arr[i] and arr[largest]

Call HEAPIFY(arr, n, largest)

UNIT 4



1. Pre order Traversal



Rule : Root → Left → Right

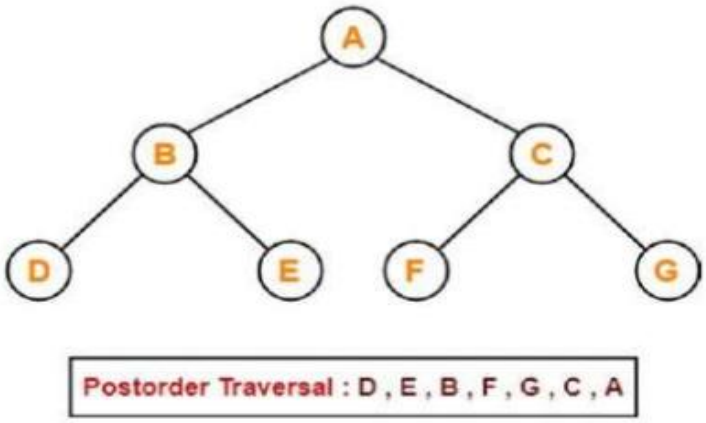
```
PREORDER(node):
    if node ≠ NULL:
        visit(node)
        PREORDER(node.left)
        PREORDER(node.right)
```

2. In order Traversal

Rule: Left → Root → Right

```
INORDER(node):
    if node ≠ NULL:
        INORDER(node.left)
        visit(node)
        INORDER(node.right)
```

3. Post order Traversal



Rule: Left → Right → Root

```
POSTORDER(node):
    if node ≠ NULL:
        POSTORDER(node.left)
```

```
    POSTORDER(node.right)
    visit(node)
```

4. Binary Search Tree Implimentation

```
INSERT(root, key):
  if root == NULL:
    create new node with key
    return new node
  if key < root.data:
    root.left = INSERT(root.left, key)
  else if key > root.data:
    root.right = INSERT(root.right, key)
  return root
```

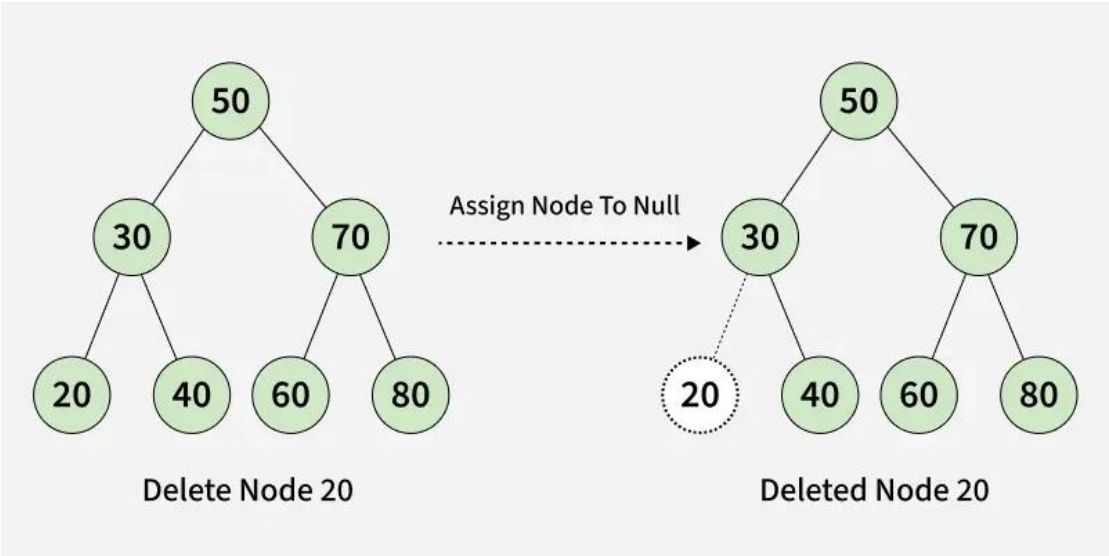
Searching in BST

```
SEARCH(root, key):
  if root == NULL or root.data == key:
    return root
  if key < root.data:
    return SEARCH(root.left, key)
  else:
    return SEARCH(root.right, key)
```

Deleting in BST

```
DELETE(root, key):
   if root == NULL:
      return root
   if key < root.data:
      root.left = DELETE(root.left, key)
   else if key > root.data:
      root.right = DELETE(root.right, key)
   else:
      // Node found
      // Case 1 & 2
      if root.left == NULL:
         return root.right
      else if root.right == NULL:
         return root.left
      // Case 3
      temp = MIN_VALUE_NODE(root.right)
      root.data = temp.data
      root.right = DELETE(root.right, temp.data)
   return root

   MIN_VALUE_NODE(node):
      current = node
      while current.left != NULL:
         current = current.left
      return current
```



5. AVL Tree rotation

RR Rotation:- RIGHT_ROTATE(y):

x = y.left

T2 = x.right

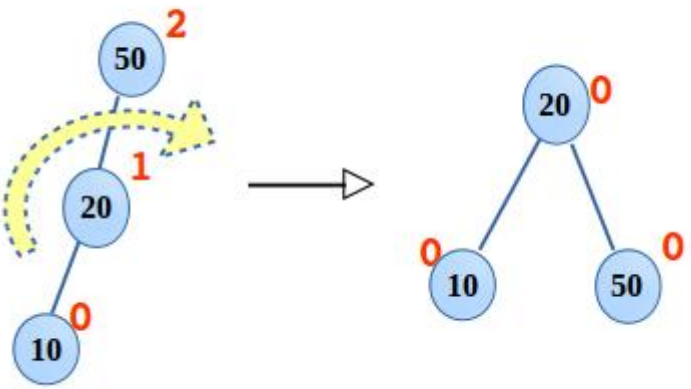
x.right = y

y.left = T2

update height of y

update height of x

return x



LL Rotation:- LEFT_ROTATE(x):

y = x.right

T2 = y.left

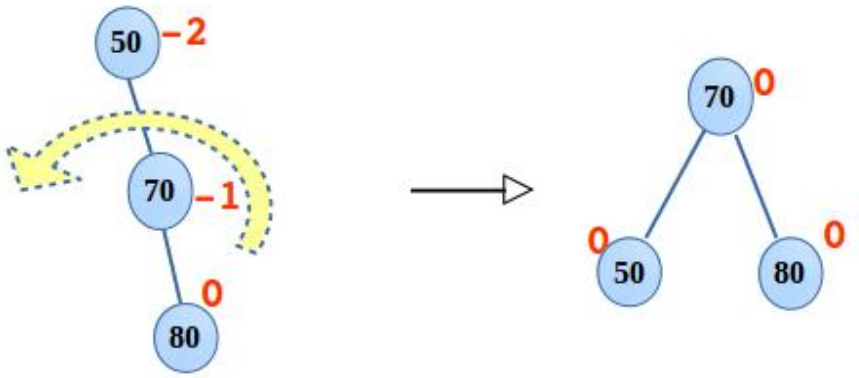
y.left = x

x.right = T2

update height of x

update height of y

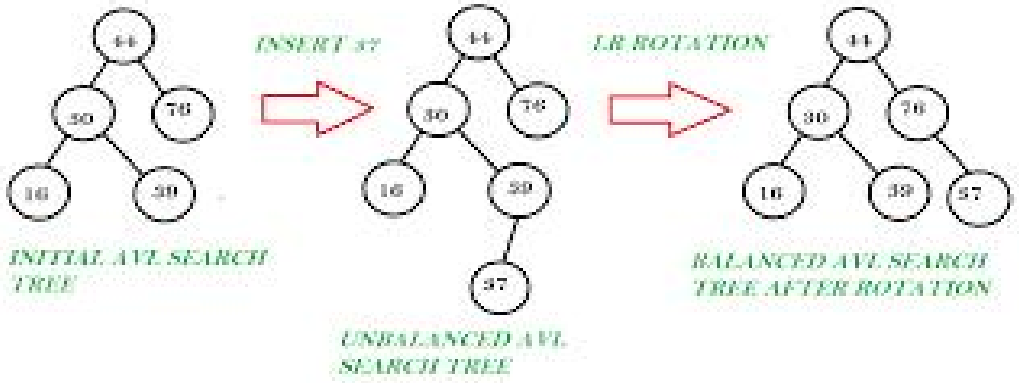
return y



LR_ROTATION(z):

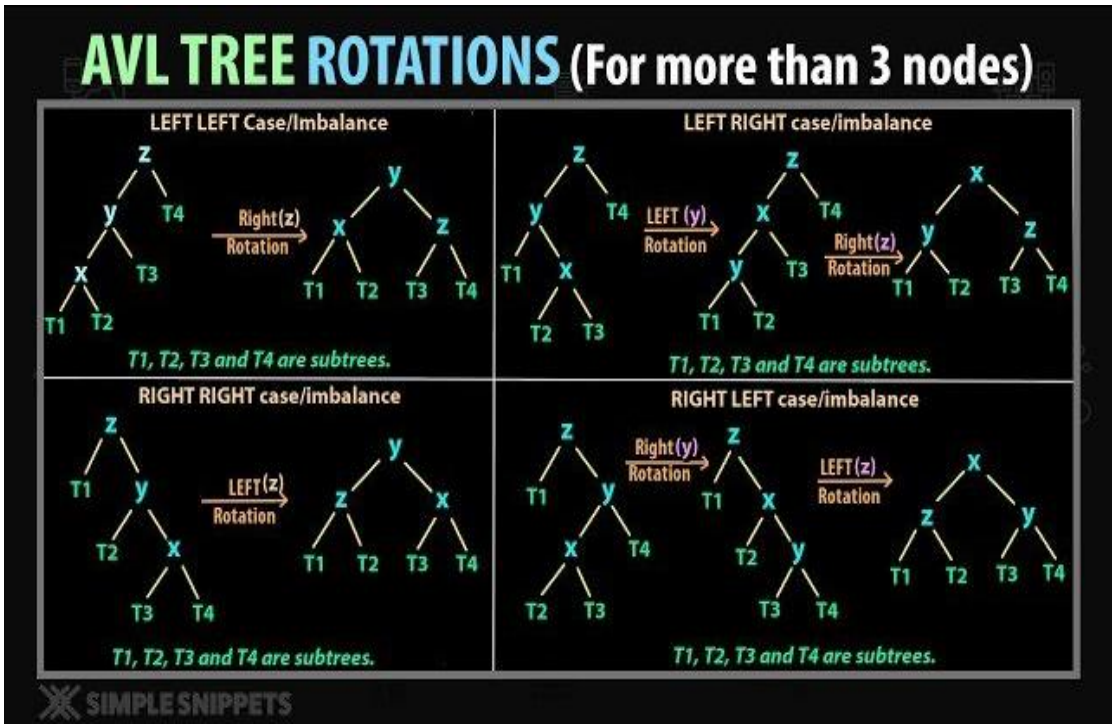
z.left = LEFT_ROTATE(z.left)

return RIGHT_ROTATE(z)



RL_ROTATION(z):

```
z.right = RIGHT_ROTATE(z.right)
return LEFT_ROTATE(z)
```



6. AVL Tree Insertion

```
INSERT(node, key):
    if node == NULL:
        return newNode(key)
    if key < node.data:
        node.left = INSERT(node.left, key)
    else if key > node.data:
        node.right = INSERT(node.right, key)
    update node.height
    balance = height(left) - height(right)
    // LL Case
```

```
    if balance > 1 and key < node.left.data:
        return RIGHT_ROTATE(node)
    // RR Case
    if balance < -1 and key > node.right.data:
        return LEFT_ROTATE(node)
    // LR Case
    if balance > 1 and key > node.left.data:
        node.left = LEFT_ROTATE(node.left)
        return RIGHT_ROTATE(node)
    // RL Case
    if balance < -1 and key < node.right.data:
        node.right = RIGHT_ROTATE(node.right)
        return LEFT_ROTATE(node)
    return node
```

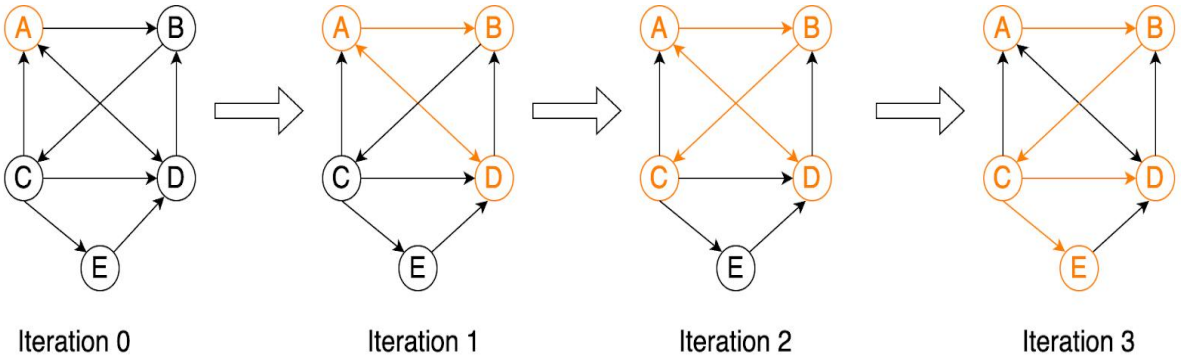
7. AVL Tree Insertion

```
DELETE(root, key):
    if root == NULL:
        return root
    if key < root.data:
        root.left = DELETE(root.left, key)
    else if key > root.data:
        root.right = DELETE(root.right, key)
    else:
        // Node found
        if root.left == NULL:
```

```
            return root.right
        else if root.right == NULL:
            return root.left
        temp = MIN_VALUE_NODE(root.right)
        root.data = temp.data
        root.right = DELETE(root.right, temp.data)
    if root == NULL:
        return root
    update root.height
    balance = height(left) - height(right)
    // LL Case
    if balance > 1 and BF(root.left) >= 0:
        return RIGHT_ROTATE(root)
    // LR Case
    if balance > 1 and BF(root.left) < 0:
        root.left = LEFT_ROTATE(root.left)
        return RIGHT_ROTATE(root)
    // RR Case
    if balance < -1 and BF(root.right) <= 0:
        return LEFT_ROTATE(root)
    // RL Case
    if balance < -1 and BF(root.right) > 0:
        root.right = RIGHT_ROTATE(root.right)
        return LEFT_ROTATE(root)
    return root
```

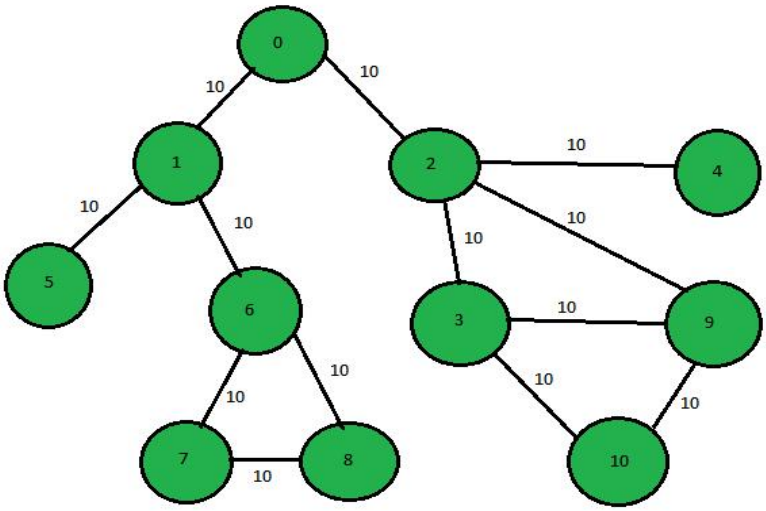
UNIT 4

1. BFS Algorithm




```
BFS(G, start):
    for each vertex v in G:
        visited[v] = false
    create empty queue Q
    visited[start] = true
    enqueue(Q, start)
    while Q is not empty:
        v = dequeue(Q)
        visit(v)
        for each adjacent vertex u of v:
            if visited[u] == false:
                visited[u] = true
                enqueue(Q, u)
```

2. DFS Algorithm



DFS(G, v):

visited[v] = true

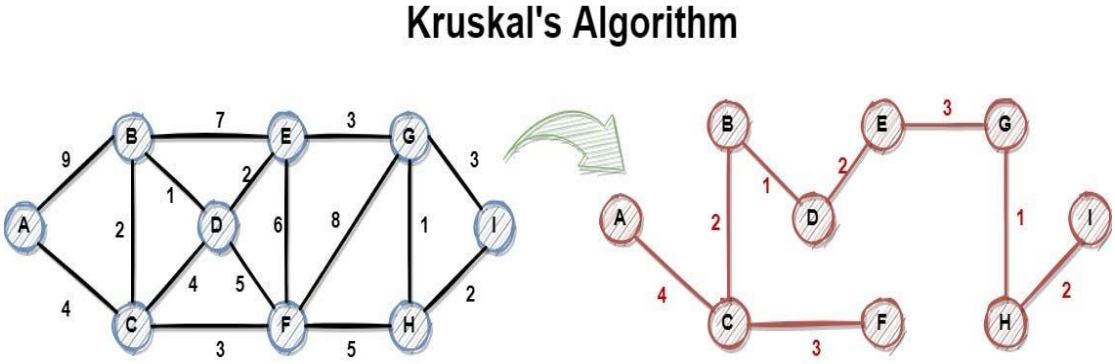
visit(v)

for each adjacent vertex u of v:

if visited[u] == false:

DFS(G, u)

3. Kruskal Algorithm



Algorithm:

KRUSKAL(G):

A = ∅

For each vertex v ∈ G.V:

MAKE-SET(v)

For each edge (u, v) ∈ G.E ordered by increasing order by weight(u, v):

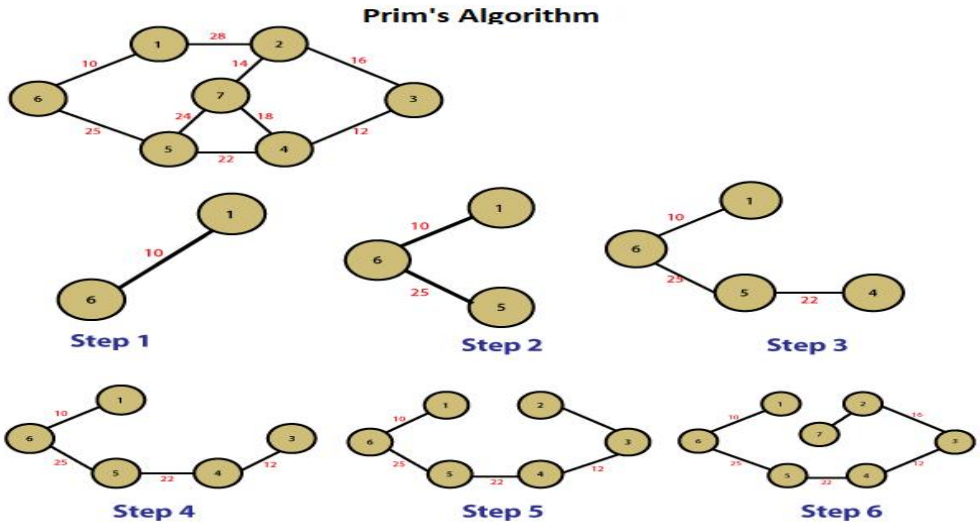
if FIND-SET(u) ≠ FIND-SET(v):

A = A ∪ {(u, v)}

UNION(u, v)

return A

4. Prims Algorithm



PRIM(G, start):

for each vertex v in G:

key[v] = ∞

parent[v] = NIL

inMST[v] = false

key[start] = 0

priorityQueue PQ

insert (start, 0) into PQ

while PQ is not empty:

u = extractMin(PQ)

inMST[u] = true

for each adjacent vertex v of u:

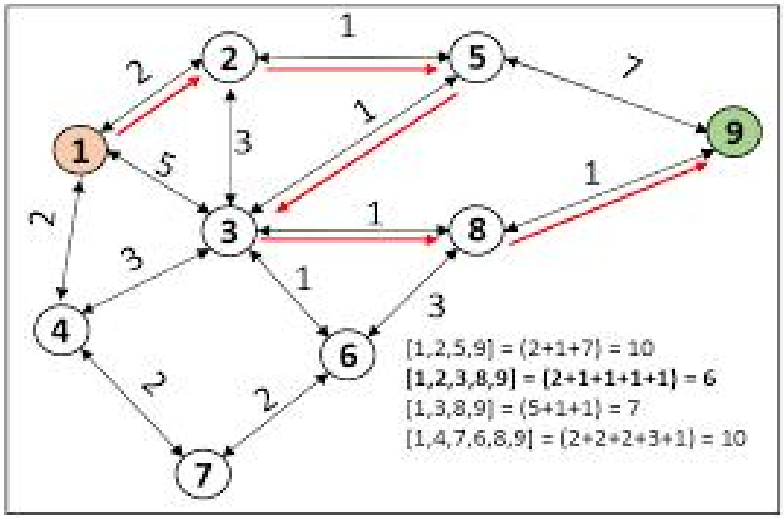
if inMST[v] == false and weight(u,v) < key[v]:

key[v] = weight(u,v)

parent[v] = u

insert/update (v, key[v]) in PQ

5. Dijkstra Algorithm



DIJKSTRA(G, source):

for each vertex v in G:

dist[v] = ∞

parent[v] = NIL

visited[v] = false

dist[source] = 0

priorityQueue PQ

insert (source, 0) into PQ

```
while PQ is not empty:
    u = extractMin(PQ)
    if visited[u] == true:
        continue
    visited[u] = true
    for each edge (u, v) with weight w:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            parent[v] = u
            insert/update (v, dist[v]) in PQ
```